

FlashRFCL: Demonstration-Efficient Sparse Reward RL via Reverse-Forward Curriculum Learning in High-Throughput GPU Simulation

Anonymous Author(s)

Affiliation

Address

email

Abstract: Solving complex tasks using reinforcement learning (RL) from sparse rewards and demonstrations is appealing since one can avoid reward shaping. However, past approaches often require heavy curriculum engineering, extensive training time, and/or infeasible amounts of demonstrations. In this paper, we investigate how scalable RL algorithms, high-throughput GPU simulations, and demonstration-based auto-curricula can address these challenges. We show that our method, FlashRFCL, can solve a variety of high-dimensional robotic tasks faster in addition to relying on just a single demonstration each. We perform extensive ablation studies on the design choices of FlashRFCL, highlighting how each component impacts training speed and performance compared to prior work.

Keywords: Reinforcement Learning, Learning from Demonstrations

1 Introduction

Reinforcement learning (RL) has shown success in robot learning for training models to high success and low cycle times, whether it is via fine-tuning Vision-Language-Action models [1] or RL followed by sim-to-real transfer. While imitation learning remains popular, scaling RL sim-to-real does not require massive demonstration datasets. Instead, with appropriate modelling of a task and dense reward shaping scaling RL has led to successes in real-world locomotion [2, 3, 4] and manipulation [5, 6, 7, 8, 9]. Many of the recent sim-to-real advances can be attributed to the high-throughput data collection speeds of GPU-parallelized simulators such as Isaac [10] and Mujoco Warp [11].

Although GPU-parallelized simulation enables inexpensive large-scale data generation, many existing approaches still rely on carefully shaped dense rewards for each task and can easily lead to suboptimal policies due to reward hacking [12]. Training with sparse rewards provides a more scalable alternative to dense reward shaping, but often requires prohibitively large amounts of exploration and long training times for tasks of practical interest. To overcome the exploration problem in sparse reward settings, online learning from demonstrations (LfD) is a promising direction to guide RL exploration toward successful behaviors. Since collecting large demonstration datasets is infeasible for complex robotics tasks, methods that can generalize from limited demonstrations to cover the task’s behavior space are highly desirable. Inverse reward learning methods such as GAIL [13] and different flavors of off-policy RL with demonstrations [14, 15, 16] fall under this category. However these methods tend to be unstable or require very long training times.

In this work, we introduce FlashRFCL, a fast and demonstration efficient LfD algorithm that leverages aggressive state-reset based curricula and large-scale GPU simulation. Specifically, we extend and adapt Reverse Forward Curriculum Learning (RFCL) [17] for GPU simulation to create our FlashRFCL algorithm. We swap out the base RL algorithm in RFCL from SAC to FlashSAC [18], and introduce key changes to hyperparameters and curriculum design to better suit the sparse reward and demonstrations setting. Through experiments in simulated robot manipulation tasks, we

37 demonstrate that FlashRFCL consistently outperforms baselines in wall time and demonstration ef-
 38 ficiency. We further perform analysis into how FlashRFCL scales to better understand how exactly
 39 the massive parallel data collection regime impacts training. Our ablations further demonstrate how
 40 our curriculum design changes improve the robustness of FlashRFCL to the number of parallel en-
 41 vironments, making it performant in both low and high-parallelism settings.

42 2 Related Work

43 **Wall-Time Efficient Reinforcement Learning Algorithms:** In massively parallel simulation set-
 44 tings where environment interactions are relatively inexpensive, the primary focus naturally shifts
 45 from sample efficiency to wall-clock training time. In this regime, on-policy RL algorithms such as
 46 PPO have achieved widespread success across domains including games and robotics, largely due
 47 to their stability and ease of scaling across distributed environments [19, 20, 3]. However, on-policy
 48 RL algorithms are inherently sample-inefficient as they require data to be collected from the cur-
 49 rent policy and discard past experiences after each update. In contrast, off-policy RL algorithms
 50 can leverage heterogeneous data sources, such as expert demonstrations or trajectories from prior
 51 policies, enabling significantly greater data reuse. Despite this advantage, off-policy RL algorithms
 52 are rarely used in massively parallel control tasks. Recent work, however, has shown that with im-
 53 proved algorithmic design, off-policy RL algorithms can scale to match or even exceed the final
 54 performance of PPO while achieving faster convergence in wall-clock time [18, 2, 21, 22, 23, 24].
 55 Similar to past work we use off-policy RL in GPU simulations, with the key difference being that
 56 we tackle sparse reward problems and leverage demonstration data.

57 **Curriculum Learning:** Curriculum learning is a training strategy that presents training examples
 58 in a structured order of increasing difficulty, enabling faster and more stable learning [25, 26, 27].
 59 Simulation enables curriculum learning through direct resets to states that are difficult to reach under
 60 the current policy but useful for learning. Prior work has leveraged this through state-reset curric-
 61 ula, where the initial state distribution is adapted during training to improve exploration. Some
 62 approaches uniformly reset to states from demonstrations [28] or past experiences [29]. Reverse
 63 curricula are another approach where policies start learning from states near goals and work back-
 64 wards [30]. Our work is an extension of Reverse Forward Curriculum Learning (RFCL) [17] that
 65 tackles sparse reward robotics tasks with few demonstrations and constructs automatic curricula
 66 out of demonstration states. We extend and adapt the RFCL algorithm to the massively parallel
 67 regime and analyze how design choices in curricula and RL impact performance. Our contributions
 68 significantly improve both the wall-time and demonstration efficiency compared to RFCL.

69 3 Preliminaries

70 3.1 Reinforcement Learning

71 We study reinforcement learning in a Markov Decision Process (MDP) defined by the tuple
 72 $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$ where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $p(s' | s, a)$ denotes the (unknown)
 73 transition dynamics, $r(s, a)$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor. The objec-
 74 tive is to learn a policy $\pi_\theta(a|s)$ parameterized by θ that maximizes the expected discounted return,
 75 $\max_\theta \mathbb{E}_{\pi_\theta} [\sum_{t=0}^{\infty} \gamma^t r_t]$. In this work, we focus on sparse reward functions, where $r(s, a) = 1$ if
 76 the next state is a success state and 0 otherwise. To mitigate exploration challenges arising from
 77 sparse rewards in high-dimensional tasks, we assume access to a dataset of N demonstrations
 78 $\mathcal{D}_{demo} = \{\tau_1, \dots, \tau_N\}$, where each demonstration $\tau_i = \{s_{i,0}, a_{i,0}, \dots, s_{i,T_i-1}, a_{i,T_i-1}, s_{i,T_i}\}$ is a
 79 trajectory of length T_i consisting of a sequence of states and actions.

80 3.2 Reverse Forward Curriculum Learning (RFCL)

81 Our work is an extension of RFCL adapted to support both the low and high-parallelism data regimes
 82 of RL. RFCL is a two-stage training scheme consisting of a reverse curriculum followed by a forward

curriculum. The reverse curriculum first learns a policy that performs well near states observed in the demonstrations. The forward curriculum then generalizes this policy to the true initial state distribution $\mathcal{S}_{init}$ by prioritizing initial states with higher estimated learning potential. This section re-describes RFCL and defines key terms used throughout the paper, with a visualization of the curriculum design shown in Fig. 1.

Reverse Curriculum: At the start of training, a reverse curriculum is constructed for each demonstration τ_i by resetting the environment to states along the demonstration. Specifically, the reverse curriculum for τ_i is defined as a sequence of stages $(\rho_{T_i}^r, \rho_{T_i-\delta}^r, \dots, \rho_0^r)$ for a small reverse step size δ , r here refers to "reverse". Each stage ρ_k^r defines a sampling distribution over demonstration states $s_{i,t+k}$, with $k \sim K$ and $k \in \{0, 1, \dots, K_{max}\}$ for some maximum discrete offset K_{max} . RFCL adopts a geometric distribution for K .

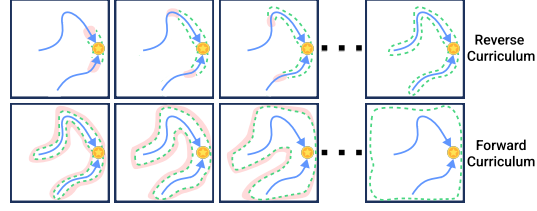


Figure 1: Original figure from RFCL. Visualization of reverse and forward curriculum progression. Blue arrows denote demonstration trajectories leading to the goal (gold star). The dashed green region represents states from which the policy achieves high return, and the red shaded region represents the most frequently sampled initial states at each stage of the curriculum.

At the beginning of each episode, RFCL samples a demonstration τ_i and an offset k , and reset the environment to state $s_{i,t+k}$. The reverse curriculum for τ_i advances to the next stage when the policy achieves success in the last m episodes initialized at state $s_{i,t}$. When ρ_0^r is reached and the policy achieves a high success rate, RFCL labels demonstration τ_i as being reverse-solved.

Forward Curriculum: After all demonstrations are reverse-solved, RFCL uniformly sample a set of n initial states from $\mathcal{S}_{init}$. These initial states are combined with the demonstration initial states Λ_{demo} to form a train set of initial states $\Lambda_{train} = \{s_{i,init}\}_{i=1}^n$ for the forward curriculum. The forward curriculum is defined as a sequence of stages $(\rho_0^f, \rho_1^f, \dots)$, where each ρ_i^f defines a sampling distribution over Λ_{train} as a convex combination of a score-prioritized distribution P_S and a staleness-prioritized distribution P_C : $P_{\rho_i^f} = (1 - \rho) \cdot P_S + \rho \cdot P_C$ where $\rho \in [0, 1]$ is a staleness coefficient.

Following PLR [26], RFCL adopts a rank-based prioritization scheme for P_S . Let q denote the fraction of successful episodes among the last k episodes initialized from $s_{i,init}$. For a success-rate threshold $\omega \in [0, 1]$, RFCL uses a simple score function that assigns a score of 2 if $q = 0$, a score of 3 if $0 < q < \omega$, and a score of 1 if $q \geq \omega$. Given scores $S = \{S_i\}_{i=1}^n$ over Λ_{train} , initial states are ranked according to S , which are then used to define the score-prioritized distribution:

$$P_S(s_{i,init} | \Lambda_{train}, S) = \frac{\text{rank}(S_i)^{-1/\beta}}{\sum_j \text{rank}(S_j)^{-1/\beta}} \quad (1)$$

where β is a temperature parameter controlling the degree of prioritization. The staleness-prioritized distribution P_C is defined analogously based on staleness scores $C = \{C_i\}_{i=1}^n$:

$$P_C(s_{i,init} | \Lambda_{train}, C, c) = \frac{c - C_i}{\sum_{C_j \in C} c - C_j} \quad (2)$$

where C_i is the episode count at which initial state $s_{i,init}$ was last sampled and c is the total number of episodes rolled out in training.

4 Method

4.1 Scaling and Robustifying RFCL

Reinforcement Learning: FlashRFCL extends RFCL by adapting to massively parallelized simulation regimes to accelerate learning in complex environments from sparse rewards. As the base RL algorithm, we swap out RFCL's use of SAC [31] with FlashSAC [18] which is better suited for GPU parallelized simulations. Notably, we adopt key changes to SAC from FlashSAC such as large-batch

updates, distributional critics, and n-step returns. Different to RFCL, the separate offline demonstration replay buffer containing demonstration transitions is optional in our method. Our experiments in Section 5.2 demonstrate how FlashRFCL can still solve tasks using the demonstration simulation states alone without needing action labels and a offline buffer.

Update-To-Data Ratio: FlashSAC achieves faster convergence by scaling model and batch sizes while reducing the update-to-data (UTD) ratio. However, in our experiments we find that low UTD ratios result in reduced wall-clock efficiency, and therefore adopt a higher UTD ratio. One reason for this discrepancy may arise from differences between sparse- and dense-reward training dynamics where more gradient updates are needed for accurate critics in low-signal settings like sparse reward tasks. Another reason is that our evaluation focuses exclusively on manipulation tasks, which prior work suggests may benefit more from higher UTD ratios [21].

Curriculum Modifications: One drawback of RFCL is that reverse curriculum learning can stall training when a particular demonstration fails to be reverse-solved, preventing transition to the forward curriculum. The motivation for a two stage split in RFCL between reverse and forward curricula was to ensure after the reverse curriculum stage RFCL would have performed enough exploration to train a somewhat robust policy. However, in GPU simulations, we find that exploration challenges caused by limited demonstrations are alleviated. Motivated by this, we construct a smoother curriculum that starts the forward curriculum once a single demonstration is reverse-solved, enabling faster expansion of state-action coverage from already reverse-solved demonstrations.

Let α denote the online/offline sampling ratio and let p_{reverse} denote the fraction of demonstrations that are not reverse-solved. For each new episode, we draw a Bernoulli variable with probability p_{reverse} to decide whether to initialize from a demonstration state (reverse curriculum) or from an initial state sampled from Λ_{train} (forward curriculum). Similarly, the probability of sampling transitions from the offline replay buffer is annealed according to $\alpha \cdot p_{\text{reverse}}$. We fix $\alpha = 0.5$ across all tasks. Consequently, offline replay transitions initially constitute 50% of training samples and are gradually phased out as the reverse curriculum completes for all demonstrations.

4.2 Improving Demonstration Efficiency

When few demonstrations are provided, the policy is overfit to a narrow state-action distribution. As a result, the policy frequently yields zero return when the simulation is initialized to most initial states in Λ_{train} during the forward curriculum. This issue is further exacerbated when using fewer environments in low parallelization regimes of RFCL, as substantially more training iterations are required to encounter states that the policy could solve the task from. To address this, we propose a simple initial-state sampling strategy that improves demonstration efficiency. We further introduce a variant of FlashRFCL with a minor algorithmic modification that can be used in conjunction with this sampling scheme, enabling effective learning from a single demonstration even with a small number of environments (e.g. 16). An algorithmic description is provided in Appendix A.

Demo-Stratified Sampling (DSS): We define states that the policy π_θ can achieve nonzero return as boundary-states. As π_θ becomes proficient over a broader state-action distribution, it increasingly encounters boundary-states, reducing the difficulty of the forward curriculum. This motivates a more fine-grained sampling strategy near $\{s_{i,\text{init}}\}_{i=1}^N$ of Λ_{demo} to preserve strong learning signals.

DSS constructs Λ_{train} by biasing the sampling towards Λ_{demo} . First, we uniformly sample $m \gg n$ candidate initial states Λ_{cand} from the true initial-state distribution $\mathcal{S}_{\text{init}}$. For each demonstration initial state in Λ_{demo} , we compute the Euclidean distance to every candidate state. To ensure balanced stratification across demonstrations, distances are normalized independently per demonstration using empirical means and standard deviations to obtain z-scores. We then define a sequence of Q increasing quantile levels $\{p_k\}_{k=1}^Q$ in the range $[0, 1]$ and compute the corresponding per-demonstration z-score quantiles $\{q_k^{(i)}\}_{i=1, k=1}^{N, Q}$. These quantiles induce strata $\{\Lambda_{\text{near-demo}}^{(k)}\}_{k=1}^Q$ over candidate states according to their normalized proximity to the demonstrations. To bias sampling toward states closer to demonstrations, we allocate target sample counts $\{N_k\}_{k=1}^Q$ using an expo-

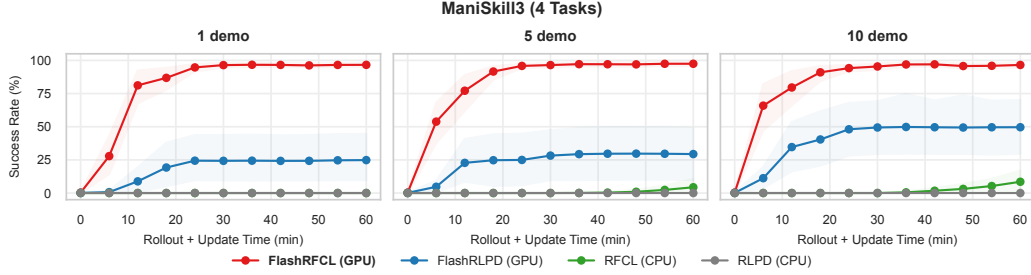


Figure 3: **Main Results.** Average success rate across all tasks given 1, 5, and 10 demonstrations. Shaded areas represent 95% CIs over 5 seeds. The results show FlashRFCL is significantly faster than prior state-of-the-art performance, solving all tasks within one hour. FlashRFCL retains both asymptotic performance and wall-time efficiency even when only a single demonstration is provided.

nentially decaying distribution over strata. We set $N + \sum_{k=1}^Q N_k = \lfloor \frac{n}{2} \rfloor$ for all tasks. Then, we uniformly sample N_k states from each stratum and combine them to obtain $\Lambda_{\text{near-demo}}$. Finally, to ensure sufficient coverage of S_{init} , we combine Λ_{demo} , $\Lambda_{\text{near-demo}}$, and an additional uniformly sampled subset of size $n - (N + \sum_{k=1}^Q N_k)$ from S_{init} to obtain Λ_{train} .

Forward Curriculum Design Choices: In the massively parallel GPU simulation regime, we find that maintaining a separate offline buffer and aggressively sampling from it (as in prior RFCL variants) is unnecessary for stabilizing forward curriculum learning. However, in settings with limited parallelism (e.g., vision-based RL or inherently low-throughput environments), we find DSS alone is sufficient but may be sample-inefficient in some tasks.

To address this, we introduce a lightweight state-reset curriculum. The key idea is to partition Λ_{train} into a feasible set $\Lambda_{\text{feasible}}$ and its complement $\Lambda_{\text{train}} \setminus \Lambda_{\text{feasible}}$, and to bias resets toward feasible states until the policy acquires sufficient state-action coverage to support forward curriculum expansion. We initialize the feasible set as $\Lambda_{\text{feasible}} = \Lambda_{\text{demo}} \cup \Lambda_{\text{near-demo}}^{(1)}$ which ensures that early rollouts are likely to achieve non-zero reward and provide meaningful learning signal. We sample initial states from the mixture distribution $P_{\text{demo-expand}} = p_{\text{demo}} \cdot P_{\text{demo}} + p_{\text{feasible}} \cdot P_{\text{feasible}} + p_{\text{explore}} \cdot P_{\text{explore}}$ where P_{feasible} is a convex combination of P_S and P_C over $\Lambda_{\text{feasible}}$ (as defined previously), and P_{demo} and P_{explore} denote uniform distributions over demonstration initial states and $\Lambda_{\text{train}} \setminus \Lambda_{\text{feasible}}$, respectively.

Let $x = \frac{|\Lambda_{\text{feasible}}|}{|\Lambda_{\text{train}}|}$. We adopt the following replay schedule:

$$p_{\text{feasible}} = 2x(1 - x), \quad p_{\text{demo}} = \max(0, 0.5 - x)(1 - p_{\text{feasible}}), \quad p_{\text{explore}} = 1 - p_{\text{demo}} - p_{\text{feasible}}$$

This schedule initially prioritizes demonstration and feasible states, providing stable learning signals in the early phase. As the feasible set expands, sampling gradually shifts away from demonstrations toward exploratory resets. Once $x > 0.5$, sampling increasingly concentrates on feasible states while reducing exploratory resets.

5 Experiments

We consider 4 challenging sparse reward robotics tasks from ManiSkill3 [32], namely PickCube, StackCube, PushT, and PegInsertionSide shown in Fig. 2. As described in the past work [17, 33], these tasks are chosen as they are more difficult than most other benchmarks due to higher initial-state randomization, variation of the geometric properties of the object, and stricter precision requirements for success.

5.1 Main Results

We evaluate FlashRFCL against 3 state-of-the-art model-free RL baselines that use demonstrations and/or simulation states: RFCL, RLPD [15], and FlashRLPD. Flash RLPD is our implementation

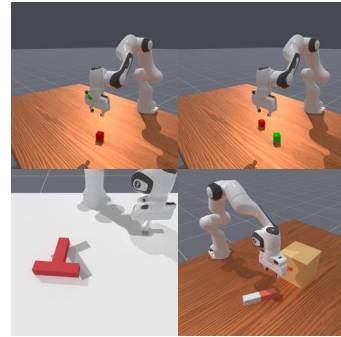


Figure 2: Visualization of tasks from ManiSkill3. From top-left to bottom-right: PickCube, StackCube, PushT, PegInsertionSide

of RLPD leveraging GPU-based simulation, with the SAC backbone replaced by FlashSAC. Both RFCL and RLPD were originally setup to train with CPU simulations, we adapt them to use the GPU simulation but with far less parallel environments to emulate the low-parallelism of CPU simulation. Results in Fig. 3 shows that FlashRFCL is the only method that reaches near 100% success rate within one hour when provided with a single demonstration. To the best of our knowledge, FlashRFCL is the first model-free RL method to solve all evaluated ManiSkill3 tasks from sparse rewards and a single demonstration. In contrast to FlashRFCL, all baselines exhibit higher sensitivity to the number of demonstrations, with the CPU simulation based algorithms being the most wall-time inefficient and needing far more demonstrations.

5.2 Training Without Demonstration Actions

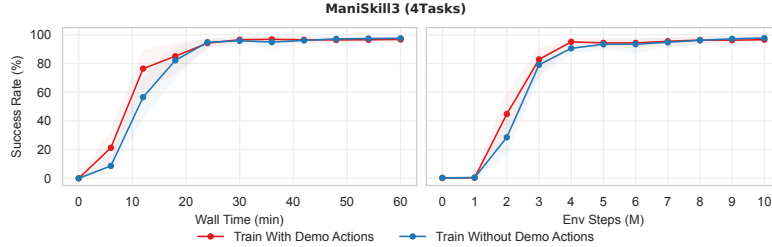


Figure 4: **Training Without Demonstration Actions.** Average success rate across all tasks when training with and without demonstration actions. Shaded areas represent 95% CIs over 5 seeds. FlashRFCL is robust to demonstration action availability, with negligible difference in wall-time and sample efficiency.

Action labels are typically tightly coupled to how the actions were collected via teleoperation or scripting, and action, whereas environment states are more reusable. Moreover, action labels may be noisy or suboptimal even when the demonstrated trajectory reaches success states. We test FlashRFCL on all tasks with and without demonstration actions, using a single demonstration per task. Without demonstration actions, we have no offline replay buffer and sample data entirely from the online replay buffer. Results in Fig. 4 demonstrate that FlashRFCL only suffers a small performance drop when we don't action labels, making FlashRFCL flexible in that it only needs simulation states and has no restrictions on how the actions that procured the simulation states are collected. For example one could train FlashRFCL to use a joint-space controller while using data collected by a teleoperator that uses an end-effector pose controller.

6 Analysis

6.1 Effect of Environment Parallelism on Reverse and Forward Curriculum

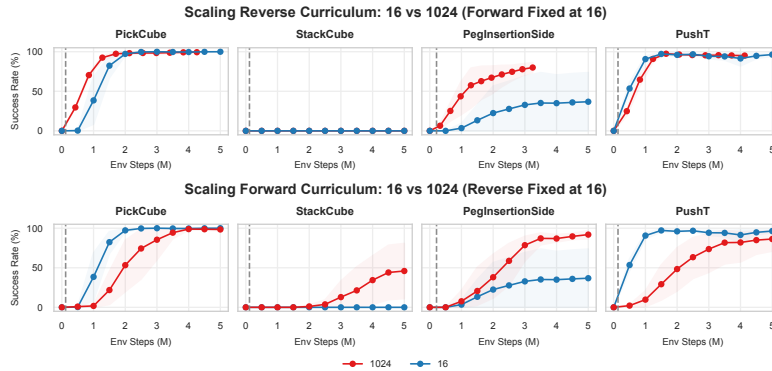


Figure 5: **Reverse to Forward.** Average success rate for each task given a single demonstration during the forward curriculum. Shaded areas represent 95% confidence intervals over 5 seeds.

We investigate the impact of environment parallelism with respect to the two curricula used in FlashRFCL. To isolate the effects of environment parallelism we disable DSS and construct Λ_{train} by uniformly sampling initial states from $\mathcal{S}_{\text{init}}$ for the forward curriculum. Fig. 5 shows the training

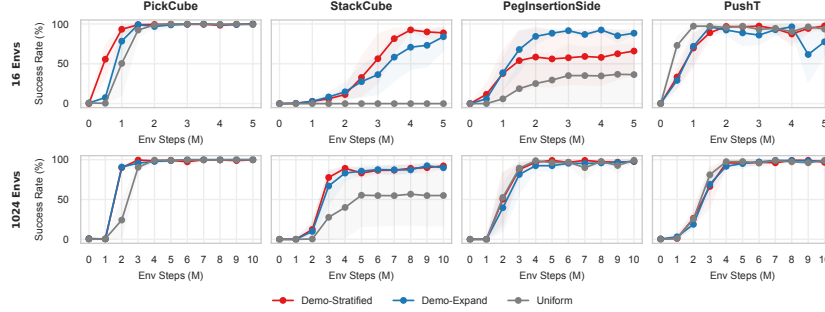


Figure 6: **Forward Curriculum Ablations.** Average success rate for each task given a single demonstration. Shaded areas represent 95% CIs over 5 seeds. Demo-Stratified and Uniform denote Demo-Stratified and uniform sampling of Λ_{train} , respectively, with $\Lambda_{\text{feasible}}$ initialized as Λ_{train} . Demo-Expand denotes Demo-Stratified Sampling with $\Lambda_{\text{feasible}}$ initialized as $\Lambda_{\text{demo}} \cup \Lambda_{\text{near-demo}}^{(1)}$.

curves of FlashRFCL during the forward curriculum when ablating on the number of parallel environments used during the reverse curriculum (top row) and during the forward curriculum (bottom row). To swap the number of parallel environments to generate Fig. 5, we save a checkpoint once the forward curriculum starts, then run forward curriculum training only with our desired checkpoint.

The top row shows that when using the exact same forward curriculum settings, the trials that used 1024 parallel environments in the reverse curriculum were more sample-efficient in most tasks. The result suggests that scaling up the reverse curriculum to more parallel environments helps train better representations that generalize better once we run the forward curriculum.

The bottom row shows that when starting from the same policy weights after running the same reverse curriculum, trials that used 1024 parallel environments in the forward curriculum were capable of achieving similar or higher success rates across all tasks after 5 million samples. Note that naturally with 1024 parallel environments, sample-efficiency is worse, but wall-time efficiency is improved. The result indicates that scaling the forward curriculum helps improve the generalization abilities of FlashRFCL despite a weak initial policy that had only explored states around one demonstration during the reverse curriculum.

6.2 Forward Curriculum Ablations

We experiment with the forward curriculum by training FlashRFCL with 16 and 1024 environments. For each setting, we compare three variants: (1) forward curriculum with DSS while initializing $\Lambda_{\text{feasible}} = \Lambda_{\text{train}}$, (2) Demo-Expand, which uses DSS while initializing $\Lambda_{\text{feasible}} = \Lambda_{\text{demo}} \cup \Lambda_{\text{near-demo}}^{(1)}$, and (3) forward curriculum with uniform sampling of Λ_{train} while initializing $\Lambda_{\text{feasible}} = \Lambda_{\text{train}}$. Fig. 6 shows that training with 1024 environments is less sensitive to forward curriculum design choices than training with 16 environments. In contrast, with only 16 environments, uniform sampling of Λ_{train} often leads to low success and fails entirely on StackCube. We find that DSS substantially improves learning with a small number of environments (e.g., 16) given only a single demonstration. Furthermore, it improves training stability and sample efficiency even in the large-scale regime with 1024 environments. Uniform sampling works in most tasks, however our experiments show that DSS is necessary for improving the robustness of FlashRFCL with respect to the amount of parallelism.

6.3 Base RL Algorithm Ablations

We examine how the choice of base RL algorithm in FlashRFCL affects wall-time and sample efficiency. In addition to FlashSAC, we consider FastTD3 [21] and PPO [34], both of which are

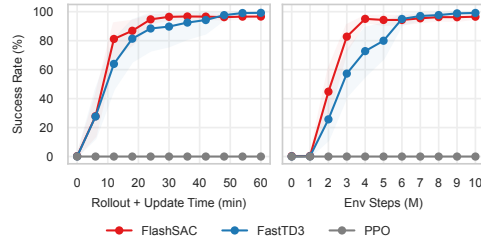


Figure 7: **Base RL Algorithm Ablations.** Average success rate of each base RL algorithm across all tasks given a single demonstration. Shaded areas represent 95% CIs over 5 seeds. FlashSAC exhibits slightly better wall-time and sample efficiency than FastTD3. We find that PPO is not suitable for our setup.

strong model-free RL algorithms well-suited for high-throughput GPU-based simulation. Because PPO is an on-policy algorithm, we use demonstrations only to reset the environment to states along the demonstration trajectories. As illustrated in Fig. 7, FlashSAC exhibits slightly better wall-time and sample efficiency over FastTD3. PPO on the other hand completely fails, indicating that the RFCL approach for sparse reward tasks with demonstrations is best used with off-policy algorithms.

6.4 Scaling Ablations

We analyze how scaling the number of environments affects the wall-time efficiency of FlashRFCL. To evaluate efficiency, we measure the wall-time required to reach a fixed performance threshold of 80% success rate. We study scaling behavior under two configurations. In the fixed-buffer setting, we use a replay buffer size of 1M transitions for all numbers of environments. In the scaled-buffer setting, we set the buffer size to 1M for 1024 environments and scale it proportionally for other environment counts to maintain a consistent buffer refresh rate. In both configurations, we observe improvements in wall-clock efficiency up to a threshold of around 1024 environments. Specifically, wall-clock efficiency increases sharply at smaller scales, followed by diminishing returns as the number of environments increases. Beyond this threshold, further increasing the number of environments leads to degraded wall-clock efficiency. This suggests that the benefits of increased data diversity from scaling the number of environments no longer compensates for the additional rollout time due to non-linear scaling of GPU simulation throughput.

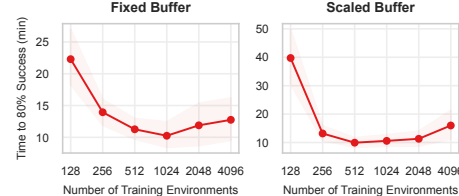


Figure 8: **Env Scaling Ablations.** Wall-clock time to reach 80% success rate vs number of environments. Results are aggregated across all tasks.

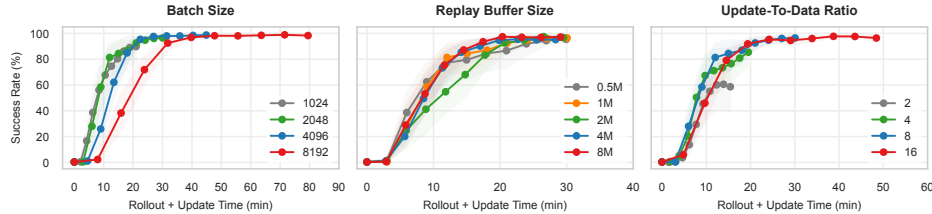


Figure 9: **Scaling Ablations.** We analyze the effects of scaling batch size, buffer size, and utd ratio. Shaded areas represent 95% CIs over 5 seeds.

Similarly to prior work, we find that increasing the batch and replay buffer size increases training efficiency by providing a more stable learning signal. However, Figure 9 indicates that increasing the batch size beyond 2048 noticeably degrades wall-clock efficiency. In contrast to FlashSAC, which found that reducing the UTD ratio to 2/1024 incurs no degradation in wall-clock efficiency or performance, Figure 9 shows that overly reducing the UTD ratio in FlashRFCL can negatively impact wall-clock efficiency.

7 Conclusion

We introduced FlashRFCL, a fast and demonstration-efficient learning-from-demonstration algorithm for high-throughput GPU simulation, capable of solving complex manipulation tasks from a single demonstration. We further introduced Demo-Stratified Sampling, which enhances the demonstration efficiency of FlashRFCL across both low- and high-parallelism settings. Through experiments we show that FlashRFCL substantially improves on both wall-time efficiency and demonstration efficiency, and provide analysis on how parallelism impacts components of the algorithm.

One limitation of FlashRFCL is reliance on resetting the environment to arbitrary states. Such resets are typically only feasible in simulation, which constrains FlashRFCL to simulated training environments. Consequently, real-world deployment of learned policy would require a sim-to-real transfer stage, but we consider this an orthogonal research direction to FlashRFCL and note that several past papers have demonstrated successful sim-to-real transfer using ManiSkill3 [8, 9, 7].

References

- [1] P. Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, J. DiCarlo, D. Driess, M. Equi, A. Esmail, Y. Fang, C. Finn, C. Glos-sop, T. Godden, I. Goryachev, L. Groom, H. Hancock, K. Hausman, G. Hussein, B. Ichter, S. Jakubczak, R. Jen, T. Jones, B. Katz, L. Ke, C. Kuchi, M. Lamb, D. LeBlanc, S. Levine, A. Li-Bell, Y. Lu, V. Mano, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, C. Sharma, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, W. Stoeckle, A. Swerdlow, J. Tan-ner, M. Torne, Q. Vuong, A. Walling, H. Wang, B. Williams, S. Yoo, L. Yu, U. Zhilinsky, and Z. Zhou. $\pi^*_{0.6}$: a VLA that learns from experience. *CoRR*, abs/2511.14759, 2025. doi: 10.48550/ARXIV.2511.14759. URL <https://doi.org/10.48550/arXiv.2511.14759>.
- [2] Y. Seo, C. Sferrazza, J. Chen, G. Shi, R. Duan, and P. Abbeel. Learning sim-to-real humanoid locomotion in 15 minutes, 2025. URL <https://arxiv.org/abs/2512.01996>.
- [3] N. Rudin, D. Hoeller, P. Reist, and M. Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning, 2022. URL <https://arxiv.org/abs/2109.11978>.
- [4] T. He, W. Xiao, T. Lin, Z. Luo, Z. Xu, Z. Jiang, J. Kautz, C. Liu, G. Shi, X. Wang, L. J. Fan, and Y. Zhu. HOVER: versatile neural whole-body controller for humanoid robots. In *IEEE International Conference on Robotics and Automation, ICRA 2025, Atlanta, GA, USA, May 19-23, 2025*, pages 9989–9996. IEEE, 2025. doi:10.1109/ICRA55743.2025.11128549. URL <https://doi.org/10.1109/ICRA55743.2025.11128549>.
- [5] P. Yin, T. Westenbroek, Z. Zhang, J. Tran, I. Dagnino, E. Shilamkar, N. Mbiziwo-Tiapo, S. Bagaria, X. Liu, G. Mullins, A. Kolobov, and A. Gupta. Emergent dexterity via diverse resets and large-scale reinforcement learning. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2603.15789>.
- [6] R. Singh, A. Allshire, A. Handa, N. D. Ratliff, and K. V. Wyk. Dextrah-rgb: Visuomotor policies to grasp anything with dexterous hands. *CoRR*, abs/2412.01791, 2024. doi:10.48550/ARXIV.2412.01791. URL <https://doi.org/10.48550/arXiv.2412.01791>.
- [7] A. Almuzairee and H. I. Christensen. Squint: Fast visual reinforcement learning for sim-to-real robotics. *arXiv preprint arXiv:2602.21203*, 2026.
- [8] A. Xie, H. Qi, and D. Sadigh. Handelbot: Real-world piano playing via fast adaptation of dexterous robot policies, 2026. URL <https://arxiv.org/abs/2603.12243>.
- [9] H.-C. Fang, A. Xie, J. Grannen, K. Llontop, and D. Sadigh. Dexdrummer: In-hand, contact-rich, and long-horizon dexterous robot drumming, 2026. URL <https://arxiv.org/abs/2603.22263>.
- [10] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State. Isaac gym: High performance gpu-based physics simulation for robot learning, 2021.
- [11] E. Todorov, T. Erez, and Y. Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012. doi:10.1109/IROS.2012.6386109.
- [12] D. Amodei, C. Olah, J. Steinhardt, P. F. Christiano, J. Schulman, and D. Mané. Concrete problems in AI safety. *CoRR*, abs/1606.06565, 2016. URL <http://arxiv.org/abs/1606.06565>.
- [13] J. Ho and S. Ermon. Generative adversarial imitation learning. In D. D. Lee, M. Sugiyama, U. von Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems 29: Annual Conference on Neural Information Processing Systems 2016, December 5-10, 2016, Barcelona, Spain*,

- pages 4565–4573, 2016. URL <https://proceedings.neurips.cc/paper/2016/hash/cc7e2b878868cbae992d1fb743995d8f-Abstract.html>.
- [14] P. Lancaster, N. Hansen, A. Rajeswaran, and V. Kumar. Modem-v2: Visuo-motor world models for real-world robot manipulation. *arXiv preprint arXiv:2309.14236*, 2023.
- [15] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine. Efficient online reinforcement learning with offline data, 2023. URL <https://arxiv.org/abs/2302.02948>.
- [16] T. Mu, M. Liu, and H. Su. Drs: Learning reusable dense rewards for multi-stage tasks. In *The Twelfth International Conference on Learning Representations*, 2024.
- [17] S. Tao, A. Shukla, T.-k. Chan, and H. Su. Reverse forward curriculum learning for extreme sample and demonstration efficiency in rl. In *International Conference on Learning Representations (ICLR)*, 2024.
- [18] D. Kim, Y. Lee, M. Park, K. Kim, I. M. A. Nahendra, T. Seno, S. Min, D. Palenicek, F. Vogt, D. Kragic, J. Peters, J. Choo, and H. Lee. Flashsac: Fast and stable off-policy reinforcement learning for high-dimensional robot control, 2026. URL <https://arxiv.org/abs/2604.04539>.
- [19] OpenAI, :, C. Berner, G. Brockman, B. Chan, V. Cheung, P. Debiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse, R. Józefowicz, S. Gray, C. Olsson, J. Pachocki, M. Petrov, H. P. d. O. Pinto, J. Raiman, T. Salimans, J. Schlatter, J. Schneider, S. Sidor, I. Sutskever, J. Tang, F. Wolski, and S. Zhang. Dota 2 with large scale deep reinforcement learning, 2019. URL <https://arxiv.org/abs/1912.06680>.
- [20] OpenAI, I. Akkaya, M. Andrychowicz, M. Chociej, M. Litwin, B. McGrew, A. Petron, A. Paino, M. Plappert, G. Powell, R. Ribas, J. Schneider, N. Tezak, J. Tworek, P. Welinder, L. Weng, Q. Yuan, W. Zaremba, and L. Zhang. Solving rubik’s cube with a robot hand, 2019. URL <https://arxiv.org/abs/1910.07113>.
- [21] Y. Seo, C. Sferrazza, H. Geng, M. Nauman, Z.-H. Yin, and P. Abbeel. Fasttd3: Simple, fast, and capable reinforcement learning for humanoid control. *arXiv:2505.22642*, 2025.
- [22] Z. Li, T. Chen, Z.-W. Hong, A. Ajay, and P. Agrawal. Parallel q -learning: Scaling off-policy reinforcement learning under massively parallel simulation, 2023. URL <https://arxiv.org/abs/2307.12983>.
- [23] A. Raffin. Getting sac to work on a massive parallel simulator: An rl journey with off-policy algorithms. *araffin.github.io*, Feb 2025. URL <https://araffin.github.io/post/sac-massive-sim/>.
- [24] A. Shukla. Speeding up sac with massively parallel simulation. <https://arthshukla.substack.com>, Mar 2025. URL <https://arthshukla.substack.com/p/speeding-up-sac-with-massively-parallel>.
- [25] Y. Bengio, J. Louradour, R. Collobert, and J. Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning, ICML ’09*, page 41–48, New York, NY, USA, 2009. Association for Computing Machinery. ISBN 9781605585161. doi:10.1145/1553374.1553380. URL <https://doi.org/10.1145/1553374.1553380>.
- [26] M. Jiang, E. Grefenstette, and T. Rocktäschel. Prioritized level replay, 2021. URL <https://arxiv.org/abs/2010.03934>.
- [27] J. Parker-Holder, M. Jiang, M. Dennis, M. Samvelyan, J. Foerster, E. Grefenstette, and T. Rocktäschel. Evolving curricula with regret-based environment design, 2023. URL <https://arxiv.org/abs/2203.01302>.

- 395 [28] A. Nair, B. McGrew, M. Andrychowicz, W. Zaremba, and P. Abbeel. Overcoming explo-
 396 ration in reinforcement learning with demonstrations, 2018. URL [https://arxiv.org/abs/](https://arxiv.org/abs/1709.10089)
 397 [1709.10089](https://arxiv.org/abs/1709.10089).
- 398 [29] A. Ecoffet, J. Huizinga, J. Lehman, K. O. Stanley, and J. Clune. Go-explore: a new approach
 399 for hard-exploration problems. *CoRR*, abs/1901.10995, 2019. URL [http://arxiv.org/](http://arxiv.org/abs/1901.10995)
 400 [abs/1901.10995](http://arxiv.org/abs/1901.10995).
- 401 [30] C. Florensa, D. Held, M. Wulfmeier, M. Zhang, and P. Abbeel. Reverse curriculum generation
 402 for reinforcement learning, 2018. URL <https://arxiv.org/abs/1707.05300>.
- 403 [31] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy
 404 deep reinforcement learning with a stochastic actor, 2018. URL [https://arxiv.org/abs/](https://arxiv.org/abs/1801.01290)
 405 [1801.01290](https://arxiv.org/abs/1801.01290).
- 406 [32] S. Tao, F. Xiang, A. Shukla, Y. Qin, X. Hinrichsen, X. Yuan, C. Bao, X. Lin, Y. Liu, T. kai Chan,
 407 Y. Gao, X. Li, T. Mu, N. Xiao, A. Gurha, V. N. Rajesh, Y. W. Choi, Y.-R. Chen, Z. Huang,
 408 R. Calandra, R. Chen, S. Luo, and H. Su. Maniskill3: Gpu parallelized robotics simulation and
 409 rendering for generalizable embodied ai. *Robotics: Science and Systems*, 2025.
- 410 [33] A. L. Escoriza, N. Hansen, S. Tao, T. Mu, and H. Su. Multi-stage manipulation with
 411 demonstration-augmented reward, policy, and world model learning, 2025. URL [https:](https://arxiv.org/abs/2503.01837)
 412 [//arxiv.org/abs/2503.01837](https://arxiv.org/abs/2503.01837).
- 413 [34] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization
 414 algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.

Algorithm 1 FlashRFCL

▷ Demo-Stratified Sampling

function DSS

 Sample candidate states $\Lambda_{\text{cand}} \sim \mathcal{S}_{\text{init}}$

 Compute per-demonstration normalized pairwise distances (z-scores) between Λ_{demo} and Λ_{cand}

 Partition Λ_{cand} into strata $\{\Lambda_{\text{near-demo}}^{(k)}\}_{k=1}^Q$ using the quantiles

 Assign each state in Λ_{cand} to a stratum using per-demonstration quantiles $\{q_k^{(i)}\}_{i=1, k=1}^{N, Q}$

 Construct $\Lambda_{\text{near-demo}}$ by uniformly sampling N_k states from each stratum

 Sample $\Lambda_{\text{uniform}} \sim \mathcal{S}_{\text{init}}$

$\Lambda_{\text{train}} \leftarrow \Lambda_{\text{demo}} \cup \Lambda_{\text{near-demo}} \cup \Lambda_{\text{uniform}}$

return Λ_{train}

▷ Training loop

while not converged **do**

 ▷ Rollout

 Sample $a \sim \pi_\theta(\cdot|s)$ given the current state s , and take action a

 Observe next state s' and reward r

 Store (s, a, r, s') in replay buffer

 ▷ Curriculum

if episode from $s_{i, \text{init}}$ is done **then**

if $\text{rand}() < p_{\text{reverse}}$ **then**

 Sample new demo id j and discrete offset value k

 Sample $s_{j, t+k}$ from τ_j in $\mathcal{D}_{\text{demo}}$

else

if $|\Lambda_{\text{train}} \setminus \Lambda_{\text{feasible}}| > 0$ **then**

 Add $s_{i, \text{init}}$ to $\Lambda_{\text{feasible}}$ **if** $\text{return}(s_{i, \text{init}}) > 0$

 Sample $s_{j, \text{init}} \sim P_{\text{demo-expand}}(s | \Lambda_{\text{train}})$

else

$s_{j, \text{init}} \sim (1 - \rho) \cdot P_S(s_{j, \text{init}} | \Lambda_{\text{train}}, S) + \rho \cdot P_C(s_{j, \text{init}} | \Lambda_{\text{train}}, C, c)$

 Reset environment to $s_{j, \text{init}}$

 ▷ Update

 Sample a replay batch from $\mathcal{D} \cup \mathcal{D}_{\text{demo}}$ with offline sampling probability $\alpha \cdot p_{\text{reverse}}$

 Perform standard off-policy RL updates

return π_θ

415 A Algorithmic Details

416 Algorithm 1 provides pseudocode for DSS and FlashRFCL.

417 B Reinforcement Learning

418 Table 1 lists all hyperparameters for FlashRFCL.

Hyperparameter	Value
FlashSAC	
Discount factor (γ)	0.95 (PickCube and StackCube) 0.99 (PegInsertionSide and PushT)
Common	
Parallel environments	1024
Replay buffer capacity	1M
Batch size	2048
Update-to-data (UTD) ratio	8/1024
TD steps (n-step)	3 (PickCube, PegInsertionSide, and PushT) 1 (StackCube)
Actor	
Number of blocks	2
Hidden dimension	128
Update delay	2
Critic	
Number of blocks	2
Hidden dimension	256
Target critic momemntum	0.1
Number of critics	2
Value prediction type	Categorical
Categorical Support	[0, 1]
Number of bins	101
Temperature	
Entropy target	0.1
Initial value	0.001
Noise Repeat	
Zeta distribution exponent	2
Maximum repeat limit	16
Reverse Curriculum	
Reverse Step Size (δ)	4
Per-trajectory Curriculum Buffer Size (m)	10
Demonstration Horizon to Episode Horizon Ratio (φ)	3
Forward Curriculum	
Forward Curriculum Score Threshold (ω)	0.75
Forward Curriculum Buffer Size (m)	10
Forward Curriculum Score Temperature (β)	0.1
Forward Curriculum Ranking Strategy	Rank
Forward Curriculum Staleness Coefficient	0.3
Forward Curriculum Number of Initial States (n)	1024
Demo-Stratified Sampling	
Number of candidate initial states (m)	1024 · 100
Number of quantiles (Q)	10
Exponential Decay Factor for Target Sampling Distribution	0.75

Table 1: Hyperparameters for FlashRFCL. Hyperparameters introduced by this paper are highlighted in blue, which are for Demo-Stratified Sampling.